

(M:N) <-----> Many To Many Relationship

● Many-to-Many (M:N) Relationship

In **1:N** or **N:1**, the rule is clear → the reference always goes on the **many** side.

But in **M:N**, **both sides are "many"**, so we have **two possible approaches**:

● Where to Store References in M:N?

Option 1: Store References in Arrays (Direct Approach)

👉 Each document keeps an array of references to the other side.

- Example: **Students** ↔ **Courses**

```
// studentSchema.js
const studentSchema = new mongoose.Schema({
  name: String,
  courses: [{ type: mongoose.Schema.Types.ObjectId, ref: "Course" }]
});

// courseSchema.js
const courseSchema = new mongoose.Schema({
  title: String,
  students: [{ type: mongoose.Schema.Types.ObjectId, ref: "Student" }]
```

```
});
```

- A `Student` has many `Courses`.
- A `Course` has many `Students`.
- Both collections **store arrays of references**.

✓ When to use

- Small to medium datasets.
- Relationship info is simple (just connections).
- Example: Songs ↔ Playlists.

✗ Problems

- If the relationship grows huge (e.g., thousands of students in a course), arrays become very large → performance issues.

Option 2: Use a Linking (Junction) Collection (Normalized Approach)

👉 Create a separate collection to **manage the relationship**.

- Example: **Enrollments collection** for Student ↔ Course.

```
const enrollmentSchema = new mongoose.Schema({
  student: { type: mongoose.Schema.Types.ObjectId, ref: "Student" },
  course: { type: mongoose.Schema.Types.ObjectId, ref: "Course" },
  enrolledAt: Date,
  grade: String
});
```

Now:

- **Student collection** stores only student data.
- **Course collection** stores only course data.
- **Enrollment collection** manages the M:N relationship.

✓ When to use

- Large datasets.
- Need to store extra info about the relationship (like `enrolledAt`, `role`, `grade`).
- Example:
 - Users ↔ Roles (with assigned date).
 - Doctors ↔ Hospitals (with working hours).

✗ Downside

- Requires **extra queries** or `populate()` chaining to fetch data.

● Rule of Thumb

- If relationship is small/simple → use arrays of references.
- If relationship is large/complex → use a linking collection.

CRUD OPERATIONS

📌 Scenario: Students ↔ Courses (Many-to-Many)

- A **student** can enroll in many **courses**.
- A **course** can have many **students**.

Option 1: Store references in both collections

✓ Schema Design

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  name: String,
  courses: [{ type: mongoose.Schema.Types.ObjectId, ref: "Course" }]
});

const courseSchema = new mongoose.Schema({
  title: String,
  students: [{ type: mongoose.Schema.Types.ObjectId, ref: "Student" }]
});

const Student = mongoose.model("Student", studentSchema);
const Course = mongoose.model("Course", courseSchema);
```

CRUD Operations

1. Create → Enroll student in a course

```
const student = await Student.create({ name: "Ashutosh" });
const course = await Course.create({ title: "MongoDB Advanced" });

// Add references
student.courses.push(course._id);
course.students.push(student._id);
```

```
await student.save();
await course.save();
```

2. Read → Get student with courses

```
const studentWithCourses = await Student.findById(student._id)
    .populate("courses");
console.log(studentWithCourses);
```

3. Update → Add new course for student

```
const newCourse = await Course.create({ title: "Mongoose Mastery" });
await Student.findByIdAndUpdate(student._id, {
    $push: { courses: newCourse._id }
});
await Course.findByIdAndUpdate(newCourse._id, {
    $push: { students: student._id }
});
```

4. Delete → Remove course from student

```
await Student.findByIdAndUpdate(student._id, {
    $pull: { courses: course._id }
});
await Course.findByIdAndUpdate(course._id, {
    $pull: { students: student._id }
});
```

```
});
```

Option 2: Use a Join Collection (Enrollment)

This is **cleaner** when you expect many records.

✓ Schema Design

```
const enrollmentSchema = new mongoose.Schema({
  student: { type: mongoose.Schema.Types.ObjectId, ref: "Student" },
  course: { type: mongoose.Schema.Types.ObjectId, ref: "Course" },
  enrolledAt: { type: Date, default: Date.now }
});

const Enrollment = mongoose.model("Enrollment", enrollmentSchema);
```

CRUD Operations

1. Create → Enroll student in a course

```
await Enrollment.create({
  student: student._id,
  course: course._id
});
```

2. Read → Get all courses for a student

```
const coursesForStudent = await Enrollment.find({ student: student._id })
  .populate("course");
console.log(coursesForStudent);
```

3. Update → Change course enrollment (student switches course)

```
await Enrollment.findOneAndUpdate(
  { student: student._id, course: oldCourse._id },
  { course: newCourse._id }
);
```

4. Delete → Unenroll student from a course

```
await Enrollment.findOneAndDelete({
  student: student._id,
  course: course._id
});
```

When to Use Which?

- **Option 1 (Embed refs in both collections):**
 - Good for **small scale** relationships.
 - Faster to query directly with `.populate()`.
 - Becomes messy if you have **millions of students & courses**.
- **Option 2 (Join collection):**
 - Best for **large scale many-to-many**.

- Easier to track **extra details** (like `enrolledAt`, `grade`, `status`).
- More flexible for reporting & analytics.